

IES COMERCIO
Marcos Rodríguez

GRIMLEDGER



Índice

1. Introducción
 1. Contexto del Proyecto y el Ecosistema D&D
 2. Problemática Actual
 3. Solución Propuesta: Grimledger
 2. Objetivos
 3. Especificación de Requisitos
 1. Requisitos Funcionales
 2. Requisitos No Funcionales
 4. Análisis
 1. Actores
 2. Casos de Uso
 3. Diagrama de Casos de Uso
 4. Diagrama de Clases
 5. Diagramas de Secuencia
 5. Diseño
 1. Arquitectura del Sistema
 2. Diagrama de Componentes
 3. Diseño de Base de Datos (ER)
 4. Diseño de Interfaz / Navegación
 6. Implementación
 1. Tecnologías Utilizadas
 2. Justificación Técnica
 7. Pruebas
 8. Mejoras Futuras
 9. Conclusiones
 10. Bibliografía
-

1. Introducción

1.1 Contexto del Proyecto y el Ecosistema D&D

El sector de los videojuegos y las herramientas digitales para juegos de rol de mesa (Tabletop Role-Playing Games - TTRPG) ha experimentado un crecimiento exponencial en la última década. Dungeons & Dragons (D&D), editado por Wizards of the Coast, se ha consolidado como el estándar de oro del género. La quinta edición (5e) es la más extendida por su equilibrio entre profundidad mecánica y accesibilidad para nuevos jugadores.

En este ecosistema, la gestión de contenidos (monstruos, hechizos, objetos, personajes y backgrounds) ha migrado progresivamente del papel a soluciones digitales. Herramientas como D&D Beyond, Roll20 o Foundry VTT han digitalizado buena parte de las reglas, pero siguen siendo limitadas cuando el usuario necesita crear contenido propio o mantener coherencia entre catálogos, formularios y reglas.

Grimledger nace en este contexto. Es una aplicación web orientada a la gestión y creación de contenido de D&D 5e, con un enfoque claro en personajes y contenido Homebrew. La propuesta del TFG consiste en combinar una SPA desarrollada en Angular 21 con un backend FastAPI, persistencia en MongoDB y una futura integración de Qwen para asistir la creación de personajes, monstruos, hechizos, backgrounds e items.

1.2 Problemática Actual

Los usuarios que crean contenido de D&D, especialmente personajes y material Homebrew, se enfrentan a una carga cognitiva inmensa. Deben gestionar reglas, catálogos, estadísticas, relaciones entre entidades y consistencia narrativa. Las herramientas actuales suelen ser pasivas: el usuario debe saber exactamente qué buscar, qué valores introducir y cómo mantener el equilibrio entre fantasía y mecánica.

Además, la gestión de personajes, monstruos, hechizos, backgrounds e items suele repartirse entre fuentes distintas o formularios poco conectados entre sí. No existe una solución unificada que permita consultar contenido, editar entidades propias y, al mismo tiempo, recibir asistencia contextual para completar formularios complejos con datos coherentes.

1.3 Solución Propuesta: Grimledger

Grimledger nace como una respuesta técnica a estas problemáticas. Es una Single Page Application (SPA) construida con **Angular 21**, apoyada por un backend en **FastAPI** y una base de datos **MongoDB**. A diferencia de herramientas centradas solo en consulta, Grimledger combina catálogo, formularios ricos y una futura capa de asistencia inteligente. En su alcance final de TFG, el proyecto incluye:

- **Gestión Centralizada:** Catálogo y edición de personajes, monstruos, hechizos, backgrounds e items desde una única aplicación.
- **Creación Guiada:** Formularios ricos con validación y estructura de dominio, incluyendo un wizard de creación de personajes.
- **Asistencia con Qwen:** Integración prevista de un modelo de la familia Qwen alojado en AWS EC2 para asistir la creación y enriquecimiento de entidades.
- **Apoyo Visual Progresivo:** Soporte para imágenes reales cuando existan y, mientras tanto, textos descriptivos o prompts de imagen para ilustraciones pendientes.

La arquitectura de Grimledger se divide en tres piezas principales: (1) el frontend en Angular 21 con Signals y componentes standalone, (2) el backend en FastAPI con Pydantic para validación tipada, y (3) MongoDB como persistencia documental. Sobre este núcleo se incorporará Qwen como runtime de asistencia especializado con PDFs originales y datos estructurados del dominio.

2. Objetivos

El objetivo general de este Trabajo de Fin de Grado es **diseñar, desarrollar y validar una aplicación web para la gestión avanzada de contenidos de Dungeons & Dragons 5ª edición, incorporando asistencia inteligente basada en Qwen para apoyar la creación de contenido original (Homebrew) y personajes.**

Para alcanzar este objetivo general, se han definido los siguientes objetivos específicos:

1. **Diseñar una arquitectura coherente con el producto real:** separar claramente frontend (Angular 21), backend (FastAPI) y persistencia documental (MongoDB).
 2. **Implementar el núcleo funcional actual:** autenticación, consulta de catálogos y creación/edición de personajes, monstruos, hechizos, backgrounds e items.
 3. **Incorporar asistencia con IA:** preparar la integración de Qwen para enriquecer formularios y generar sugerencias estructuradas sobre entidades del dominio.
 4. **Especializar el contexto del modelo:** utilizar PDFs originales, base de datos propia y estructuras del dominio como base de conocimiento para el asistente.
 5. **Definir un despliegue simple y defendible:** alojar la solución en una única instancia AWS EC2, priorizando control del stack y viabilidad técnica.
-

3. Especificación de Requisitos

3.1 Requisitos Funcionales

Los requisitos funcionales definen QUÉ debe hacer el sistema. A continuación se enumeran los requisitos funcionales de Grimledger, agrupados por módulos:

1. Gestión de Usuarios (RF01-RF03):

- **RF01 - Registro de usuario:** El sistema debe permitir a cualquier usuario crear una cuenta proporcionando email, contraseña (mínimo 8 caracteres, una mayúscula, un número) y nombre de usuario. La contraseña se almacena hasheada con bcrypt.
- **RF02 - Inicio de sesión:** El sistema debe validar credenciales y emitir un JWT (JSON Web Token) con validez de 24 horas para acceder a las rutas protegidas.
- **RF03 - Gestión de roles:** El sistema debe diferenciar entre usuario estándar y administrador, restringiendo edición y administración según permisos.

2. Catálogo de D&D 5e (RF04-RF07):

- **RF04 - Visualización y filtrado:** El sistema debe permitir listar y consultar monstruos, hechizos, backgrounds, items, razas y clases en múltiples vistas.
- **RF05 - Búsqueda y detalle:** El sistema debe permitir buscar entidades por nombre y acceder a su ficha completa con la información relevante del dominio.
- **RF06 - Soporte de catálogos:** El sistema debe reutilizar catálogos de D&D 5e para poblar formularios y vistas de detalle.
- **RF07 - Gestión de imágenes:** El sistema debe permitir usar imagen real cuando exista y, en su defecto, conservar una descripción textual de la ilustración pendiente.

3. Creación y Edición de Entidades (RF08-RF12):

- **RF08 - Gestión de personajes:** El sistema debe permitir crear personajes mediante un wizard paso a paso y persistirlos asociados al usuario autenticado.
- **RF09 - Gestión de monstruos:** El sistema debe permitir crear, editar y visualizar monstruos con estadísticas, habilidades y acciones.
- **RF10 - Gestión de hechizos y backgrounds:** El sistema debe permitir crear, editar y consultar hechizos y backgrounds con sus estructuras específicas.
- **RF11 - Gestión de items:** El sistema debe soportar la creación y edición de armas, armaduras, objetos mágicos, herramientas, monturas y equipo de aventura.
- **RF12 - Edición con control de autoría:** El sistema debe restringir la modificación de contenido según permisos y autoría.

4. Asistencia con Qwen (RF13-RF15):

- **RF13 - Asistencia sobre formularios:** El sistema debe preparar una integración de Qwen capaz de asistir la creación de personajes, monstruos, hechizos, backgrounds e items.
- **RF14 - Respuestas estructuradas:** La asistencia de IA debe devolver sugerencias estructuradas y seguras para aplicar sobre el estado local del formulario.
- **RF15 - Enriquecimiento textual y visual:** La IA debe poder proponer descripciones, ajustes de contenido y textos descriptivos de imagen cuando todavía no exista ilustración final.

3.2 Requisitos No Funcionales

Los requisitos no funcionales definen CÓMO debe comportarse el sistema en términos de calidad, rendimiento y seguridad:

- 1. **RNF01 - Rendimiento:** El sistema debe responder con fluidez en consultas de catálogo y mantener tiempos de respuesta razonables en operaciones asistidas por IA.
- 2. **RNF02 - Arquitectura modular:** El frontend y el backend deben permanecer desacoplados, permitiendo ampliar el asistente IA sin reescribir los formularios principales.

3. **RNF03 - Seguridad:** Todas las rutas protegidas deben requerir JWT válido. Las contraseñas nunca se almacenan en texto plano y la autorización debe respetar rol y autoría.
 4. **RNF04 - Eficiencia operativa:** El sistema debe poder mantenerse inicialmente sobre una única instancia AWS EC2 con consumo de recursos controlado.
 5. **RNF05 - Usabilidad:** La interfaz debe ser clara, responsive y consistente con el tono visual de Grimledger, tanto en escritorio como en tablet.
 6. **RNF06 - Portabilidad:** El backend debe poder ejecutarse en cualquier servidor con Python 3.11+ y Docker instalado. El frontend debe ser compatible con navegadores modernos (Chrome, Firefox, Edge, Safari) sin necesidad de plugins.
-

4. Análisis

4.1 Actores

En el sistema Grimledger se han identificado tres actores principales. En el contexto del análisis UML, un actor representa un rol que interactúa con el sistema para consultar información, crear entidades o administrarlas.

1. Usuario No Registrado (Visitante):

Representa a cualquier persona que accede a la aplicación sin haber iniciado sesión. Puede visualizar la home y acceder a los formularios de registro y login, pero no puede consultar entidades protegidas ni crear contenido.

2. Usuario Registrado:

Es el actor principal del sistema. Puede consultar catálogos, crear y editar sus personajes, y trabajar con monstruos, hechizos, backgrounds e items según los permisos definidos. También será el principal beneficiario de la integración de Qwen como asistencia contextual sobre formularios.

3. Administrador (Admin):

Es el actor con mayores privilegios. Además de todo lo que puede hacer un usuario registrado, puede gestionar contenido con permisos ampliados y administrar usuarios según la política de roles del sistema.

Es importante destacar que, aunque una misma persona pueda actuar como usuario registrado y administrador, en el modelo de seguridad son roles distintos y el backend verifica el claim de rol en cada petición protegida.

4.2 Casos de Uso

Los casos de uso describen las interacciones específicas entre los actores y el sistema. A continuación se detallan los casos de uso principales, agrupados por módulos. Cada caso de uso sigue el formato: "El sistema permite a [Actor] [Acción] para [Objetivo]".

1. Módulo de Autenticación:

- **CU01 - Registrar Usuario:** El sistema permite a un Usuario No Registrado crear una cuenta proporcionando email, contraseña y nombre de usuario. El sistema valida que el email no esté en uso, que la contraseña cumpla con la política de seguridad, y almacena el hash en MongoDB. Finalmente, redirige al login.
- **CU02 - Iniciar Sesión:** El sistema permite a un Usuario No Registrado (o Registrado) autenticarse mediante email y contraseña. El sistema valida las credenciales contra la base de datos, genera un JWT (firmado con una clave secreta) y lo devuelve al cliente Angular, que lo almacena en localStorage para futuras peticiones.
- **CU03 - Cerrar Sesión:** El sistema permite a un Usuario Registrado (o Admin) cerrar su sesión. El sistema invalida el JWT en el cliente (eliminandolo de localStorage) y redirige al home. (Nota: el JWT sigue siendo válido hasta su expiración a menos que se implemente una blacklist en el backend).

2. Módulo de Catálogo y Consulta:

- **CU04 - Consultar Catálogos:** El sistema permite a un Usuario Registrado listar y consultar monstruos, hechizos, items, razas, clases y backgrounds.
- **CU05 - Buscar y Filtrar:** El sistema permite buscar entidades por nombre y aplicar filtros según el tipo de contenido mostrado.
- **CU06 - Ver Detalle:** El sistema permite acceder a la ficha completa de una entidad y visualizar sus datos estructurados.
- **CU07 - Navegar entre Manual y Detalles:** El sistema permite moverse entre el catálogo/manual y las vistas detalladas sin perder contexto.

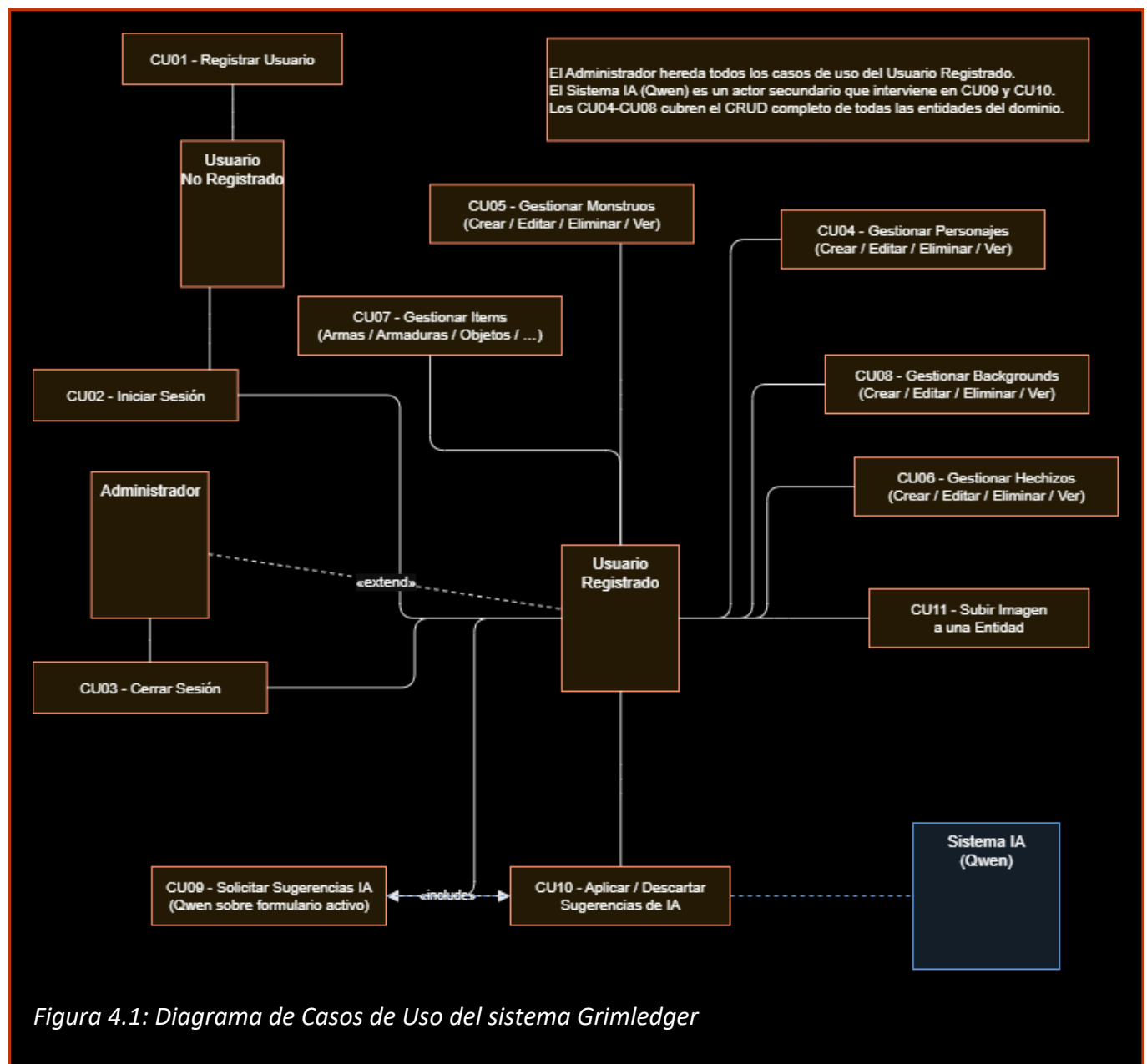
3. Módulo de Creación y Edición:

- **CU08 - Crear Personaje:** El sistema permite construir un personaje mediante un wizard con pasos de raza, clase, background, estadísticas y resumen.
- **CU09 - Crear o Editar Monstruo:** El sistema permite gestionar monstruos con estadísticas, habilidades, acciones y soporte de imagen.
- **CU10 - Crear o Editar Hechizo, Background e Item:** El sistema permite trabajar con formularios especializados según el dominio de cada entidad.

4. Módulo de Asistencia con IA:

- **CU11 - Solicitar Sugerencias a Qwen:** El sistema permite enviar contexto saneado del formulario activo a Qwen para recibir sugerencias estructuradas.
- **CU12 - Aplicar Ajustes Propuestos:** El usuario puede aceptar o descartar sugerencias de IA sobre textos, estadísticas o estructuras del formulario.
- **CU13 - Generar Descripción de Imagen:** El sistema puede proponer una descripción textual de la ilustración pendiente cuando todavía no exista un archivo real.

4.3 Diagrama de Casos de Uso



Explicación del Diagrama de Casos de Uso:

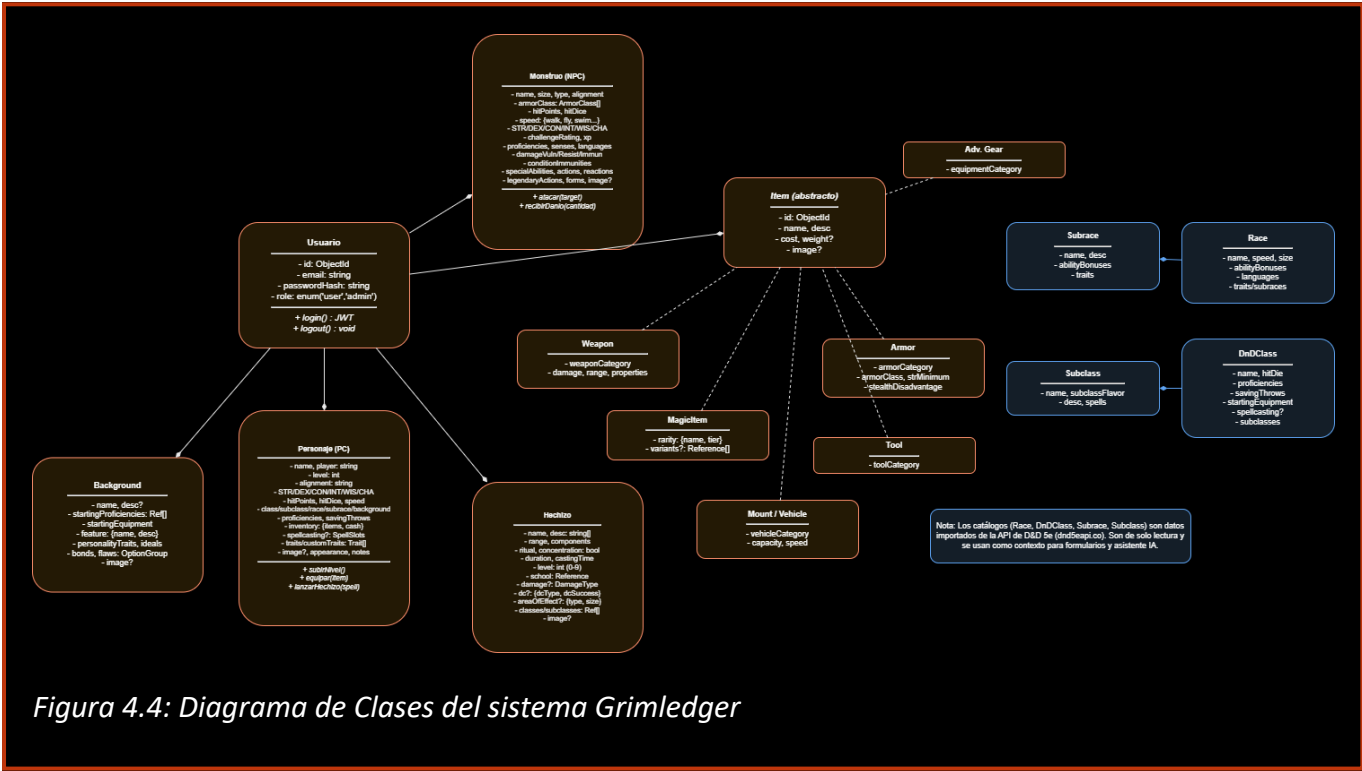
El diagrama de casos de uso (Use Case Diagram) es una herramienta visual fundamental en el análisis de requisitos. En este documento representa los actores reales del proyecto y los casos de uso vinculados al alcance final del TFG.

Relaciones principales:

- El **Usuario No Registrado** solo tiene acceso a dos casos de uso: "Registrar Usuario" (CU01) y "Iniciar Sesión" (CU02). Esto refleja la política de seguridad de "registro abierto, uso restringido".
- El **Usuario Registrado** concentra el núcleo funcional del sistema: consultar catálogos, crear y editar entidades del dominio, y utilizar la asistencia de Qwen sobre formularios.
- El **Administrador** hereda todo lo anterior y añade capacidades de gestión ampliada sobre contenido y usuarios.

Notas de diseño: En UML, los casos de uso se conectan a los actores mediante asociaciones (líneas sólidas). Si un caso de uso "incluye" obligatoriamente a otro (ej: "Generar Monstruo" incluye "Validar Descripción"), se usa una relación «include» (flecha discontinua). Si un caso de uso es opcional y depende de una condición (ej: "Exportar a PDF" extiende a "Ver Personaje"), se usa una relación «extend» (flecha discontinua con punta).

4.4 Diagrama de Clases



Explicación del Diagrama de Clases:

El diagrama de clases (Class Diagram) modela la estructura estática del sistema: clases, atributos y relaciones. En Grimledger, la clase central es **Usuario**, vinculada a entidades editables como **Personaje**, **Monstruo**, **Hechizo**, **Background** e **Item**.

Entidades principales y sus relaciones:

- **Usuario:** Atributos: -id, -email, -passwordHash, -role. Relación: 1 Usuario puede crear y gestionar sus propias entidades.
- **Personaje:** Atributos: -id, -nombre, -nivel, atributos, inventario, rasgos y spellcasting. Relación: cada personaje pertenece a 1 Usuario.
- **Monstruo:** Atributos: -id, -nombre, -challenge rating, estadísticas, acciones y habilidades.
- **Hechizo:** Atributos: -id, -nombre, -nivel, -escuela, -tiempo de lanzamiento, -rango y -componentes.

- **Background e Item:** Modelan rasgos, equipo inicial y variantes de objetos según el dominio de D&D.

Diagrama de base de datos vs Diagrama de clases: Aunque en MongoDB no usamos "tablas" tradicionales, el diagrama de clases ayuda a visualizar cómo los documentos JSON se relacionan entre sí. Por ejemplo, en MongoDB, un documento de **Usuario** podría tener un array embebido de **Personaje** (si son pocos), o referenciarlos mediante IDs (si un usuario tiene cientos).

4.5 Diagramas de Secuencia

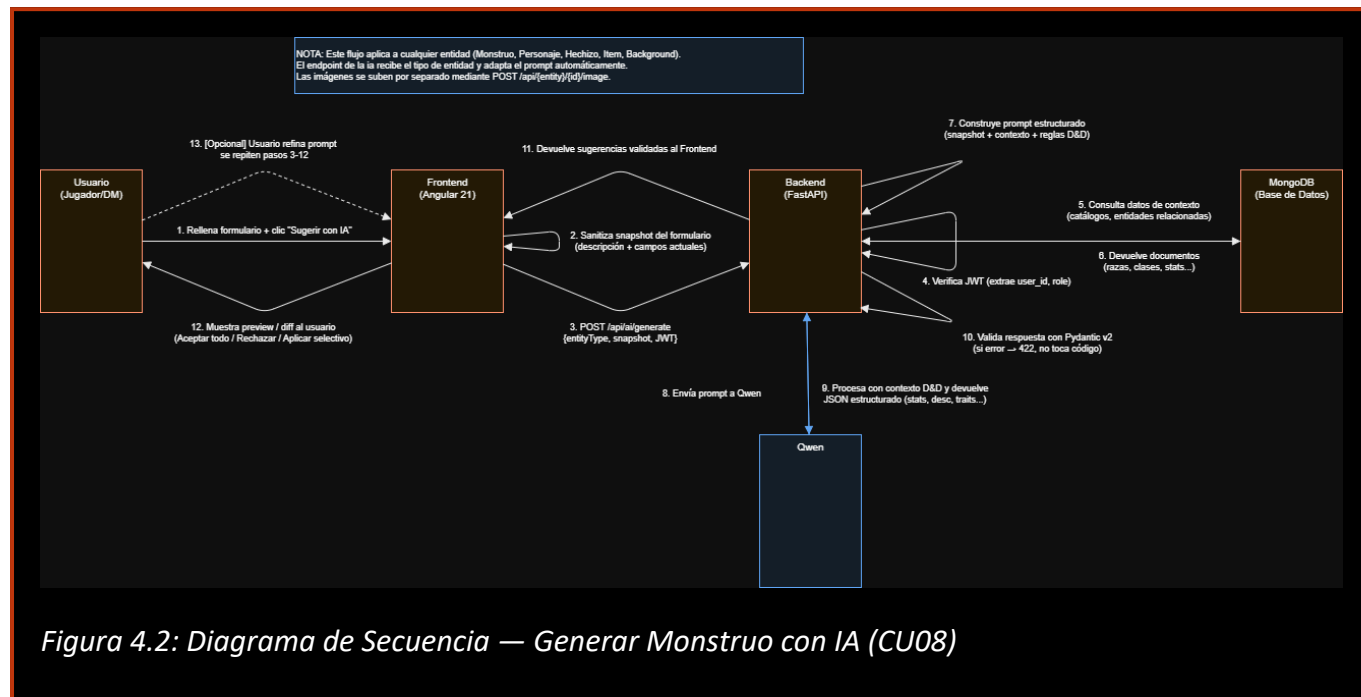
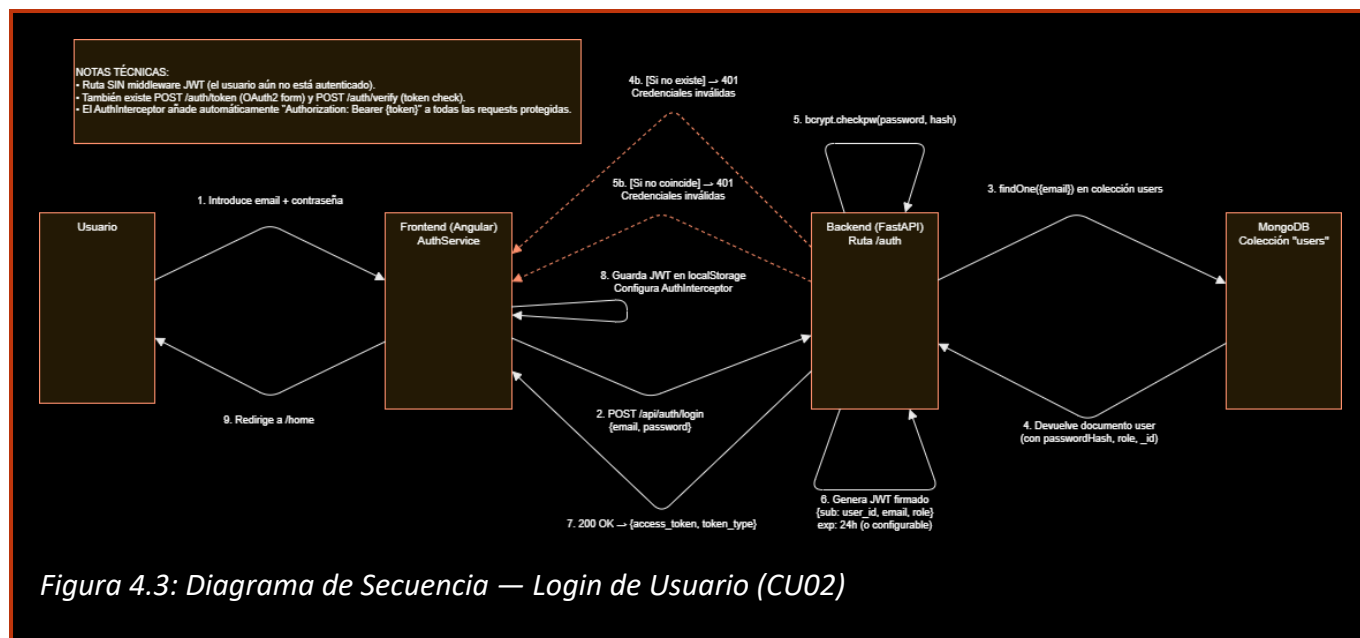


Figura 4.2: Diagrama de Secuencia — Generar Monstruo con IA (CU08)

Explicación del Diagrama de Secuencia (CU08 - Generar Monstruo):

El diagrama de secuencia (Sequence Diagram) modela cómo se intercambian mensajes entre los objetos a lo largo del tiempo para completar un caso de uso específico. Para el caso CU08 (Generar Monstruo con IA), los pasos son:

1. El **Usuario** escribe una instrucción en el formulario activo (por ejemplo, para crear un monstruo o enriquecer un personaje) y hace clic en "Generar".
2. El **Frontend (Angular)** valida el contexto mínimo, adjunta el JWT del usuario en la cabecera HTTP Authorization: Bearer <token> y envía un snapshot saneado del formulario a /api/ai/generate.
3. El **Backend (FastAPI)** recibe la petición, verifica el JWT, enriquece el contexto con datos de MongoDB y construye un prompt estructurado para Qwen.
4. El **Servicio de IA (Qwen en AWS EC2)** recibe el prompt, procesa la petición y devuelve una respuesta estructurada con sugerencias de contenido.
5. El **Backend** valida la respuesta usando Pydantic y devuelve al frontend una estructura segura y consistente.
6. El **Frontend** muestra la sugerencia, el diff o los campos propuestos para que el usuario decida si aplicarlos.



Explicación del Diagrama de Secuencia (CU02 - Login):

El caso de uso de login (CU02) es crítico porque establece la identidad del usuario para todo el sistema. El diagrama muestra:

1. El **Usuario** introduce email y contraseña en un formulario de Angular (login page).
2. Angular usa el AuthService (un servicio inyectable) para enviar POST a /api/auth/login con { "email": "...", "password": "..." }.
3. El **Backend (FastAPI)** recibe la petición (nota: esta ruta NO tiene middleware de JWT porque el usuario aún no está autenticado). Busca el usuario en **MongoDB** por email. Si existe, usa `bcrypt.checkpw()` para comparar la contraseña proporcionada con el hash almacenado.
4. Si las credenciales son válidas, el Backend genera un JWT (JSON Web Token) con payload { "sub": user_id, "email": "...", "role": "user" }, lo firma con una clave secreta (almacenada en variables de entorno), y lo devuelve al Frontend con status 200.
5. El **Frontend** recibe el JWT, lo almacena en `localStorage` (o `sessionStorage`), y redirige al usuario a la página principal (/home).

Nota técnica: A partir de este momento, todas las peticiones a rutas protegidas (cualquier cosa excepto /login y /register) deben incluir el JWT en la cabecera Authorization. El interceptor de Angular (AuthInterceptor) se encarga de añadirlo automáticamente.